

Lab 3: Architecture Justification

Topic: Personalized Meal & Diet Subscription Manager

K B BALAVIDESHNA
PES1UG24CS206
5-D

Architecture Selection

Layered Architecture is used.

The system is organised into three layers:

1. a Presentation layer (User Interface Component for subscribers and dietitians)
2. a Business layer (Authentication & Access Control, Dietary Profile Manager, Meal Plan Manager and Subscription Manager)
3. a Data layer (Database Component)

Components talk through the interfaces IAuth, IProfile, IMealPlan, ISubscription and IDataAccess.

Reasons

1. Clear separation of clinical logic from everything else
Allergen exclusion, macro-nutrient limits and dietitian approval are business rules that must be correct every time. Keeping them in the Business layer, apart from the UI and the database, lets them be tested and changed in one place without touching how subscribers or dietitians interact with the app.
2. Fits the scale and consistency needs of the scenario
This is one subscription service with five core components and highly related data. A layered design with a single central database keeps that data consistent and is far simpler to build, deploy and maintain than Microservices, whose network overhead and distributed data consistency problems are not justified here.
It also avoids the single-server bottleneck of plain Client-Server by keeping the business logic modular and replaceable.

Security Advantage

Every request from the UI must pass through the Business layer, where the Authentication & Access Control Component checks the user's role before any data is read or written.

The UI has no direct path to the Database Component, so a subscriber's allergies, diabetic limits and meal plans cannot be reached without authorisation. A single enforcement point is easier to audit than checks scattered across services.

Performance Benefit

Because all schedule changes (pause, skip, reschedule) and delivery status lookups go through one Subscription Manager to one data store, they run as simple in-process calls and single database queries, without the network hops that a Microservices design would add between services.

This helps schedule changes even under peak load, and the Business layer can be scaled horizontally behind a load balancer if demand grows.